

Superscalar processors

- Scalar processors are those which executes one instruction per cycle.
- In a superscalar processor, multiple instructions are issued per cycle and multiple results are generated per cycle.
- Superscalar processors are designed to exploit more instruction-level parallelism in user programs.
- The amount of instruction level parallelism varies widely depending on the type of code being executed.
- A superscalar processor of degree m can issue m instructions per cycle.
- To fully utilize the superscalar processor of degree m , m instructions must be executed in parallel, which may not be possible in all clock cycles. In this situation some of the pipelines may be stalling in a wait state.

*refer book page 164-167 fig 4.12 and 4.13

Superscalar Processor

- Superscalar Processors:

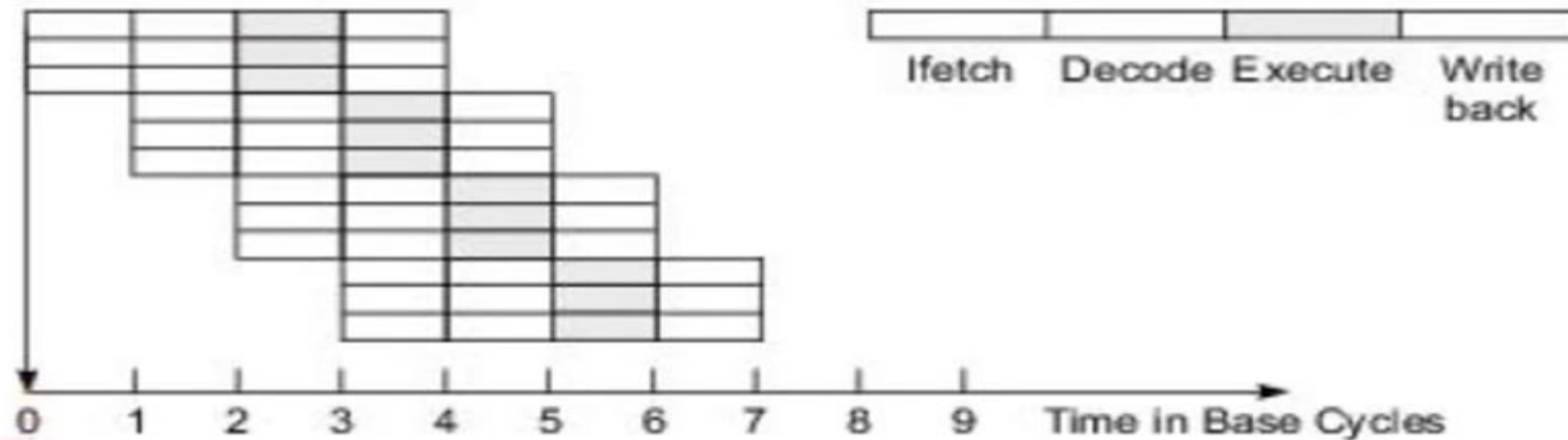
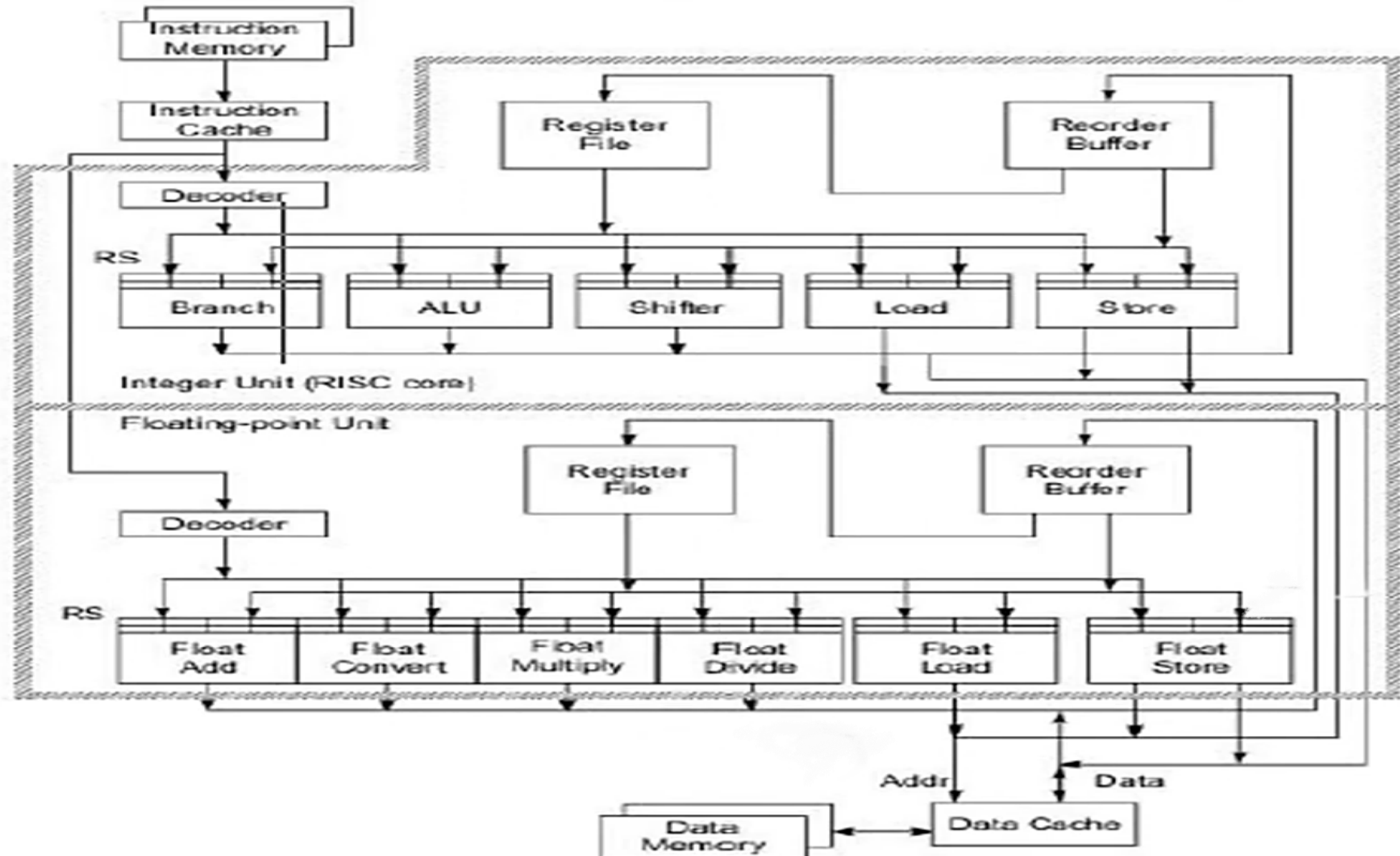


Fig. 4.11 A superscalar processor of degree $m = 3$

Superscalar RISC Processor

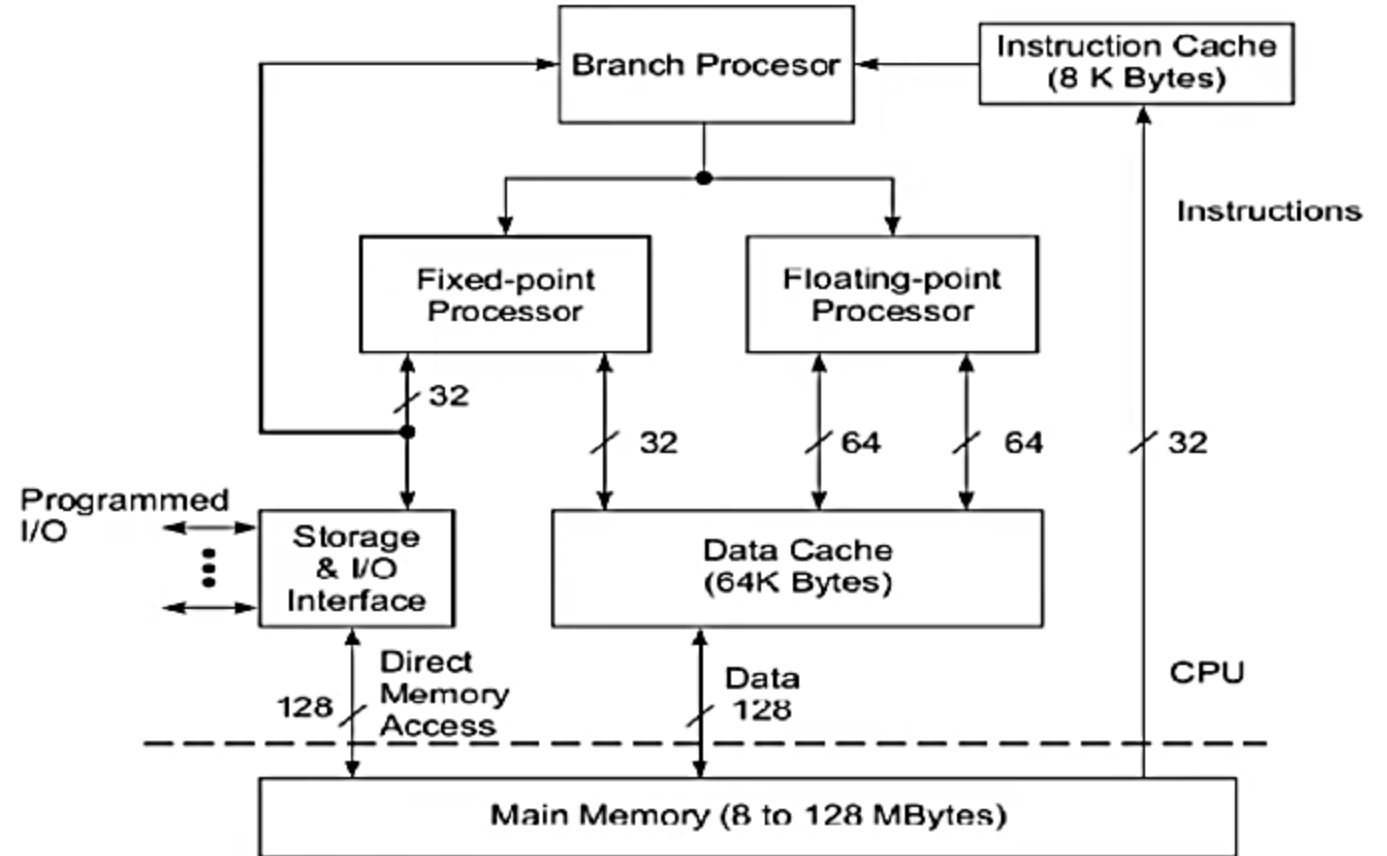


IBM RS/6000

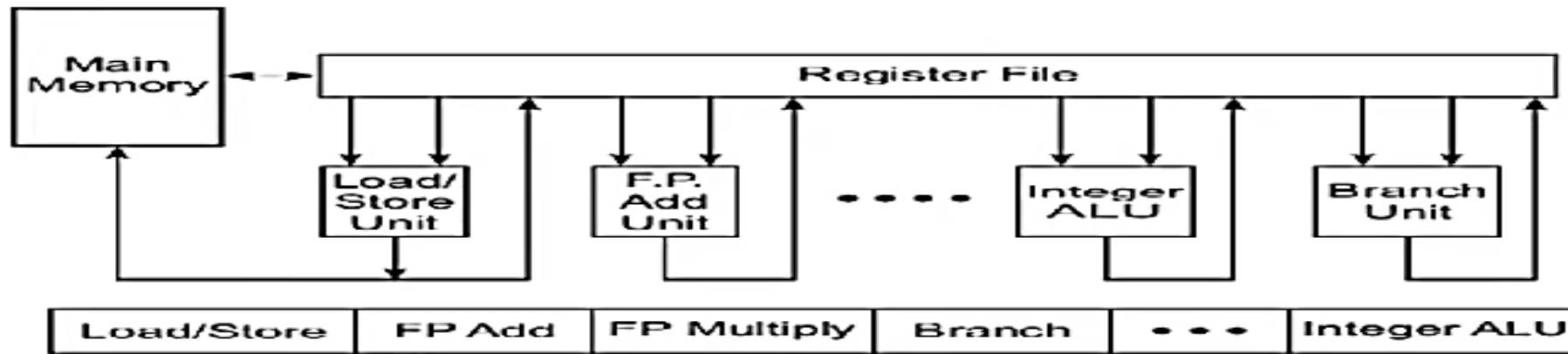
Superscalar Processor

IBM RS/6000

- 3 FUs
 - Fixed point unit
 - Floating point Unit
 - Branch processor unit - arranges execution of 5 IPC
- Hardwired control
- Wide buses



VLIW (Very Long Instruction Word)



(a) A typical VLIW processor with degree $m = 3$



(b) VLIW execution with degree $m = 3$

VLIW vs Superscalar

	VLIW	Superscalar
Decoding	Easier - Simple decoding	Complex
Code density	When ILP is lesser, code density worsens	Better code density when ILP is less
CPI (Cycles per instruction)	Lower	Better
Effectiveness	Depends on efficiency of code compaction	Dynamic (adaptive) behavior
Detecting parallelism	Requires explicit encoding of parallelism. However, No need for additional H/w or S/w to detect parallelism. [Simpler hardware design]	Complex hardware design. Requires hardware and software (compiler) support.

Vector Processors

- A vector processor executes vector instructions on arrays of data, each vector instruction involves a string of repeated operations (a vector instruction involves a large array of operands), which are ideal for pipelining with one result per cycle.
- A vector processor can be assumed a register-to-register architecture or a memory-to-memory architecture.
- Register-to-register architecture uses shorter instructions and vector register files.
- The memory-to-memory architecture uses memory based instructions which are longer in length including memory addresses.

**A vector instruction is a type of computer instruction that allows a processor to perform the same operation on multiple data elements simultaneously, essentially processing an entire array of data with a single instruction, leading to significantly improved performance for applications that involve large data sets like image processing, scientific simulations, and video encoding.

Vector Processors

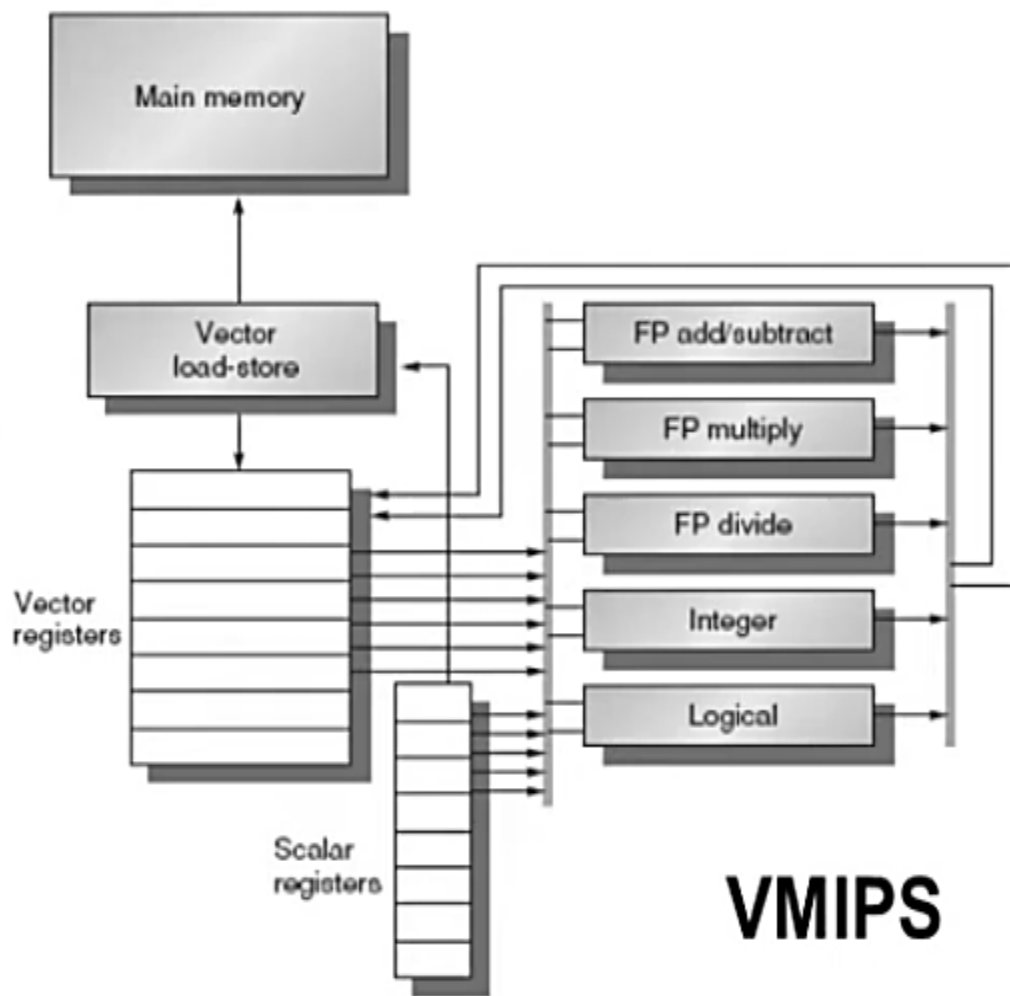
Memory to memory VP

- Memory based instructions
- Longer instructions
- Instructions include memory address

Register to register VP

- Shorter instructions
- Vector register files

The basic structure of a vector-register architecture



Vector Processors

Vector Instructions (Register to register)

- Binary Vector $\mathbf{V1} \circ \mathbf{V2} \rightarrow \mathbf{V3}$
 - Scaling $\mathbf{sI} \circ \mathbf{V1} \rightarrow \mathbf{V2}$
 - Binary Reduction $\mathbf{V1} \circ \mathbf{V2} \rightarrow \mathbf{sI}$
 - Vector Load $\mathbf{M(I:n)} \rightarrow \mathbf{V1}$
 - Vector Store $\mathbf{V1} \rightarrow \mathbf{M(I:n)}$
 - Unary Vector $\circ \mathbf{V1} \rightarrow \mathbf{V2}$
 - Unary Reduction $\circ \mathbf{V1} \rightarrow \mathbf{sI}$
-

Vector Processors

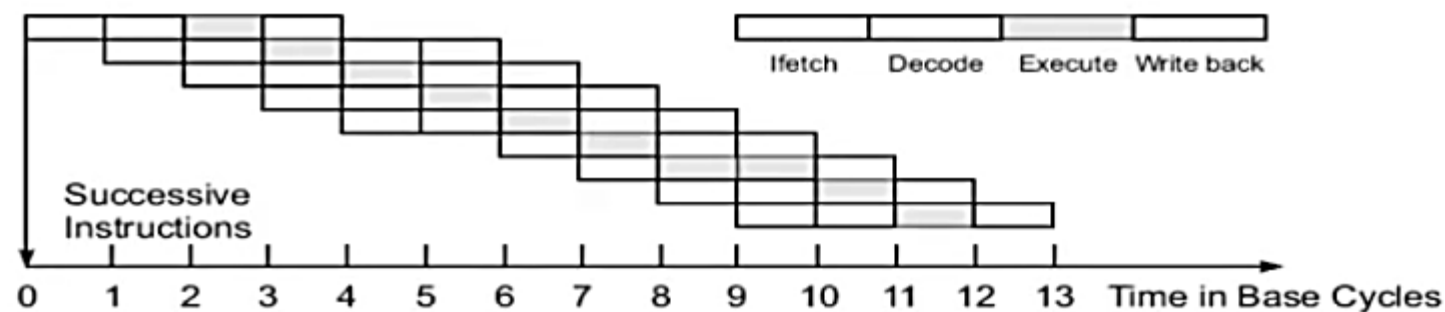
Vector Instructions (Memory to memory)

$M_1(l:n) \circ M_2(l:n) \rightarrow M_3(l:n)$

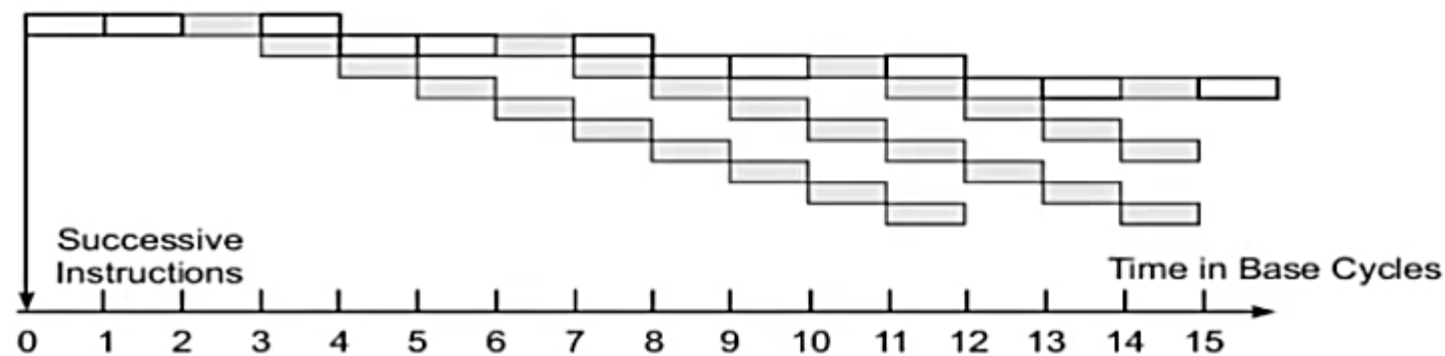
$s_1 \circ M_1(l:n) \rightarrow M_2(l:n)$

$\circ M_2(l:n) \rightarrow M(l:n)$

$M_1(l:n) \circ M_2(l:n) \rightarrow M(k)$



(a) Scalar pipeline execution (Fig. 4.2a redrawn)



(b) Vector pipeline execution

Vector Pipelines

Attached to any scalar or superscalar processor

- **Principles of vector processing**
- **Vector instructions**
- Vector instructions perform parallel operations on multiple data elements simultaneously. They are also known as single-instruction multiple-data (SIMD) instructions.
- **Vector arrays**
- A vector is an ordered set of data items in a one-dimensional array.
- **Vector processing in a pipeline**
- Data processing elements are connected in series, with the output of one being the input of the next.
- **Vector processing in an array**
- One process is broken into multiple sub-processes, and each sub-process is processed by a separate array.

Multi-vector processor and SIMD Processor

Multivector Designs

- ▶ A process that allows the CPU to execute a single instruction simultaneously on multiple pieces of data, rather than by repetitive looping.
- ▶ superscalar designs can take advantage of parallelism in scalar operations, it is possible to take advantage of similar parallelism in vector codes , Thus, it makes sense to provide multiple vector processors in a system.
- ▶ Here the main issue is memory access.

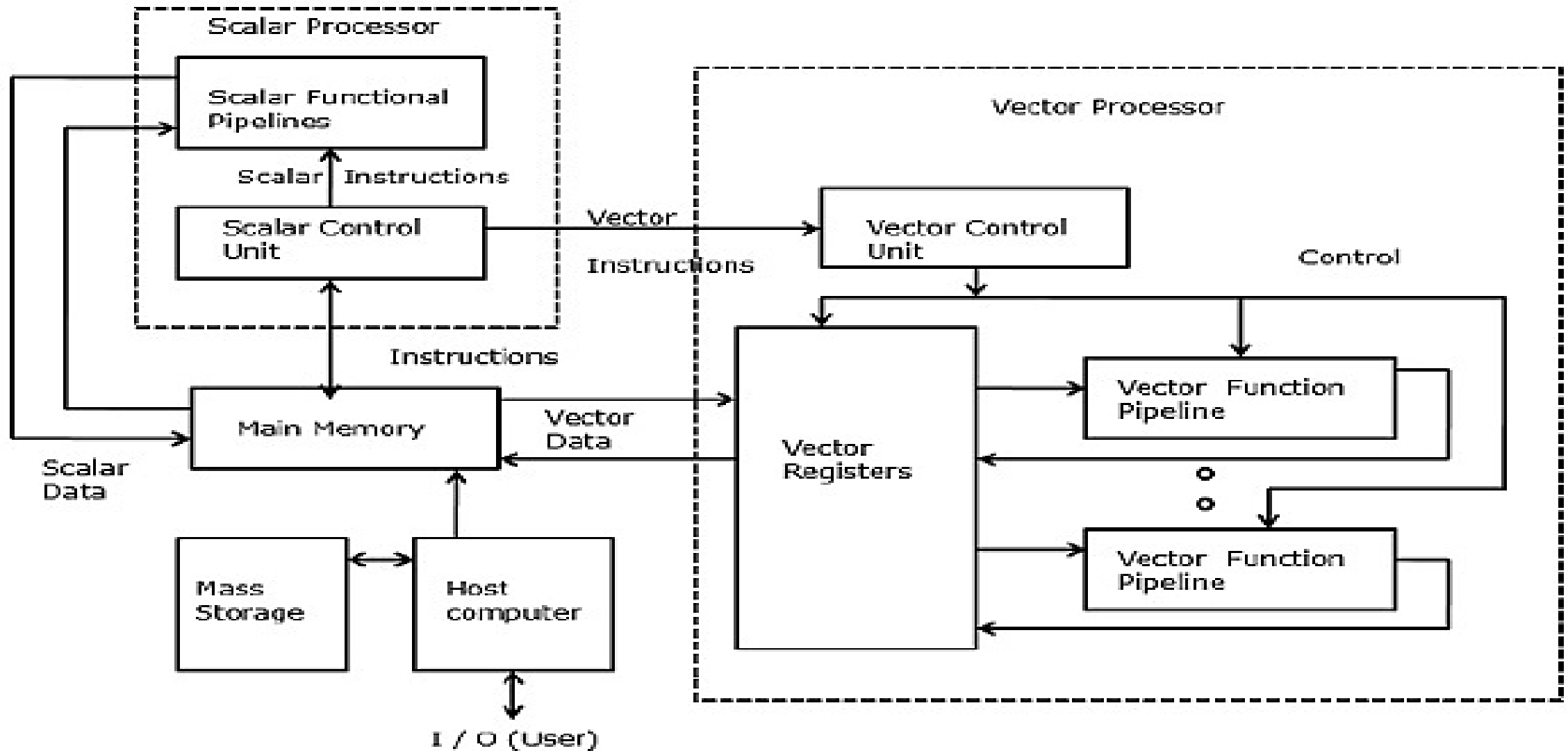
Multiprocessor

- ▶ **Multiprocessing** is the use of two or more central processing units (CPUs) within a single computer system.
- ▶ Ability of a system to support more than one processor and/or the ability to allocate tasks between them.
- ▶ The terms **multitasking** or **multiprogramming** are more appropriate to describe this concept.

Why Choose a Multiprocessor?

- ▶ A single CPU can only go so fast, use more than one CPU to improve performance
- ▶ Multiple users
- ▶ Multiple applications
- ▶ Multi-tasking within an application
- ▶ Responsiveness and/or throughput

Multiprocessors



SIMD computers

- SIMD means single instruction stream and multiple data stream.
- These computers are array processors. There are multiple processing elements which are supervised under same control unit. Each processing element receives same instruction but operate on different data from distinct module.
- In SIMD computers, 'N' number of processors are connected to a control unit and all the processors have their individual memory units. All the processors are connected by an interconnection network.

SIMD Machine Model

An operational model of an SIMD computer is specified by 5- Triple.

$$M = [N, C, I, M, R]$$

Where

(1) N is the number of processing elements (PEs) in the machine.

(2) C is the set of instructions directly executed by control unit including scalar and program flow control instructions.

(3) I is set of instructions broadcast by CPU to all PEs for parallel execution.

These include arithmetic, logic, data routing, masking and other local operations executed by each active PE over data within that PE.

(4) M is the set of masking schemes, where each mask partitions the set of PEs into enabled & disabled subsets.

(5) R is the set of data routing functions, specifying various patterns to be set up in the inter connection network for inter PE communications.

SIMD Computer:

